

MYGYMBRO

Davide Tamburrini

1 SOFTWARE REQUIREMENT SPECIFICATION

1.1 Introduction

1.1.1 Aim of the document

The purpose of this document is to present the fundamental and specific requirements of the “MyGymBro” system, a project developed for the Software Engineering and Web Design course, Academic Year 2025–2026.

The document provides a comprehensive overview of the system’s objectives, core functionalities, technical requirements, and the rationale behind the design choices, in order to guide the development process and ensure alignment among all team members

Prepared by: Davide Tamburrini

1.1.2 Overview of the defined system

The system is a Java-based software platform structured according to the MVC architecture, developed with Maven, and featuring a graphical interface created using JavaFX and FXML files. The system allows for efficient and modern management of typical gym activities, including:

- Creation of personalized training plans
- Management of the athletes by a personal trainer
- Viewing reports for each training session completed
- Viewing user progress

There are two user types with different functionalities: athletes and Personal Trainers.

1.1.3 HW e SW requirements

Software requirements:

- Java 17+
- Scene Builder (for generating FXML files)
- Draw.io (for creating UML diagrams)
- IDE: IntelliJ IDEA or Eclipse
- MySQL Server

Hardware requirements:

- CPU: Dual-Core 2.0 GHz
- RAM: 2 GB
- Disk Space: 1 GB
- Operating System: Windows, MacOS, Linux (compatible with Java 17+)

1.1.4 Releted systems

System 1: Virtuagym

Pros:

- Excellent mobile interface
- Food and fitness tracking

Cons:

- Complex web interface

System 2: Strong - Workout Tracker

Pros:

- Offers a large built-in library of exercises.
- Provides an intuitive interface for manual logging during a session.
- Allows users to share templates with others.

Cons:

- Limited Trainer Interaction
- advanced features locked behind a significant monthly subscription.

1.2 User Stories

Athlete:

- As an Athlete, I want to view the list of all my training plans, so that I am always aware of the exercises included in my personalized program.
- As an Athlete, I want to record the number of repetitions and the weight lifted for each set during a live session, so that I can accurately monitor my performance and progress over time.

Personal Trainer:

- As a personal trainer I want to edit my client's workout plans so that I can manage their programming
- As a personal trainer I want to create new workout plans so that I can assign personalized programs to my clients.

1.3 Functional requirements

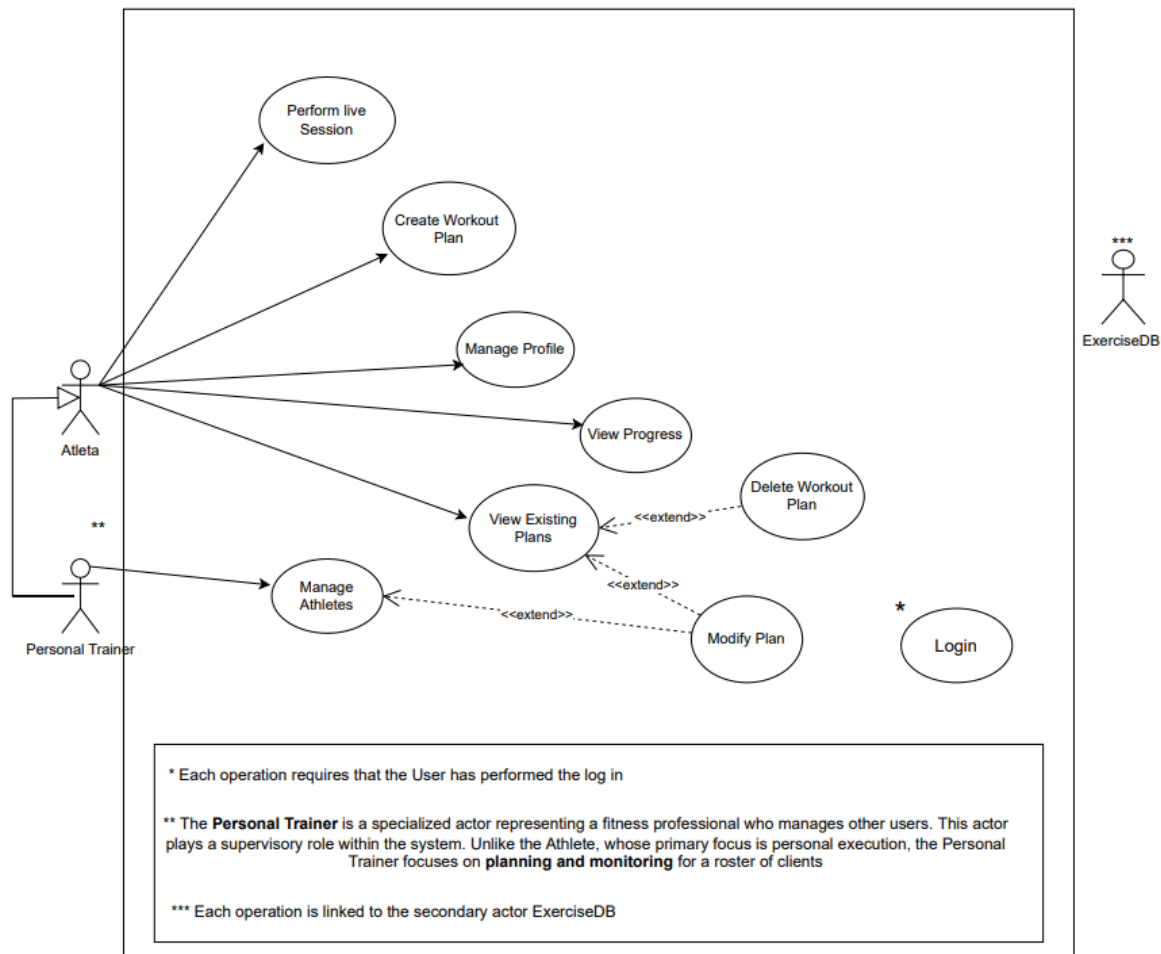
Training Plan Management:

- The system shall generate a new training plan for a client selected by the personal trainer.
- The system shall provide a workout builder interface that enables the Athlete to generate and save a personalized training plan

Reports and Statistics:

- The system shall generate and display reports and statistics related to the gym's performance.

1.4 Use Cases Diagram



1.4.1 Internal Steps

Use Case: Execute Live Session

Main flow:

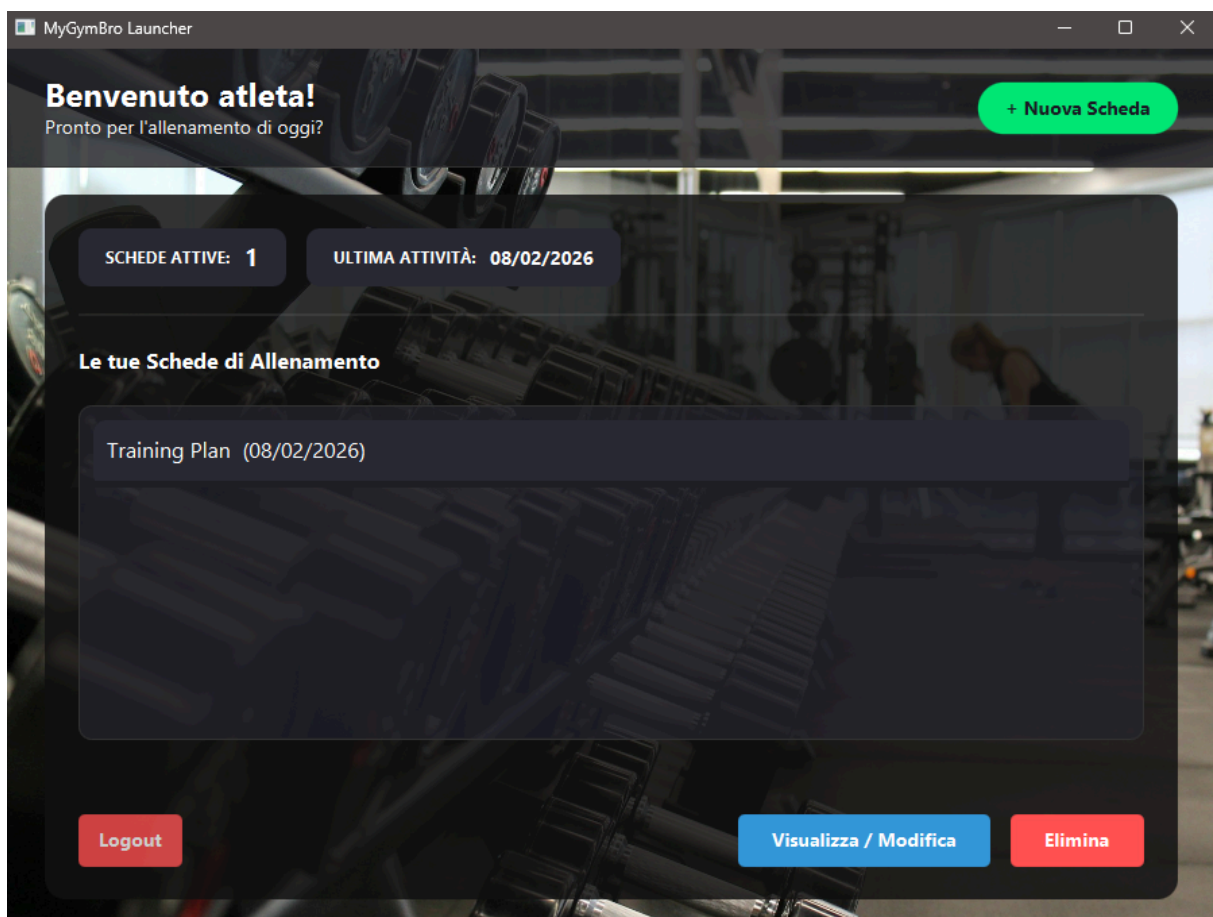
1. The Athlete initiates the live session for a selected workout plan.
2. The System displays the first exercise details, including target sets and repetitions.
3. The Athlete performs the set and inputs the actual repetitions and weight used.
4. The Athlete confirms the completion of the set.
5. The System logs the set data and starts the rest timer.
6. The System alerts the Athlete when the rest period expires.
7. The Athlete repeats the process for all sets and exercises in the plan.
8. The System displays a session summary and saves the performance data to the database.

Extensions:

- 3.a: The Athlete enters invalid or non-numeric data in the input fields.
 - 3.a.1: The System displays an error notification and prevents the confirmation of the set.
- 5.a: The Athlete chooses to skip the rest timer.
 - 5.a.1: The System terminates the timer and immediately displays the input fields for the next set.
- 8.a: The Athlete decides to terminate the session before completion.
 - 8.a.1: The System asks for confirmation and discards the session data

2 STORYBOARDS

Athlete's Home:



Workout Builder:

MyGymBro Launcher

COSTRUTTORE SCHEDA

Configura gli esercizi e i parametri di allenamento

Durata stimata: ~7 min

Nome della Scheda

Es: Massa Invernale

Note e Commenti

Note aggiuntive per l'atleta...

AGGIUNGI ESERCIZIO

Cerca esercizio...

assisted chest dip (kneeling)

Sets

Reps

Rest (s)

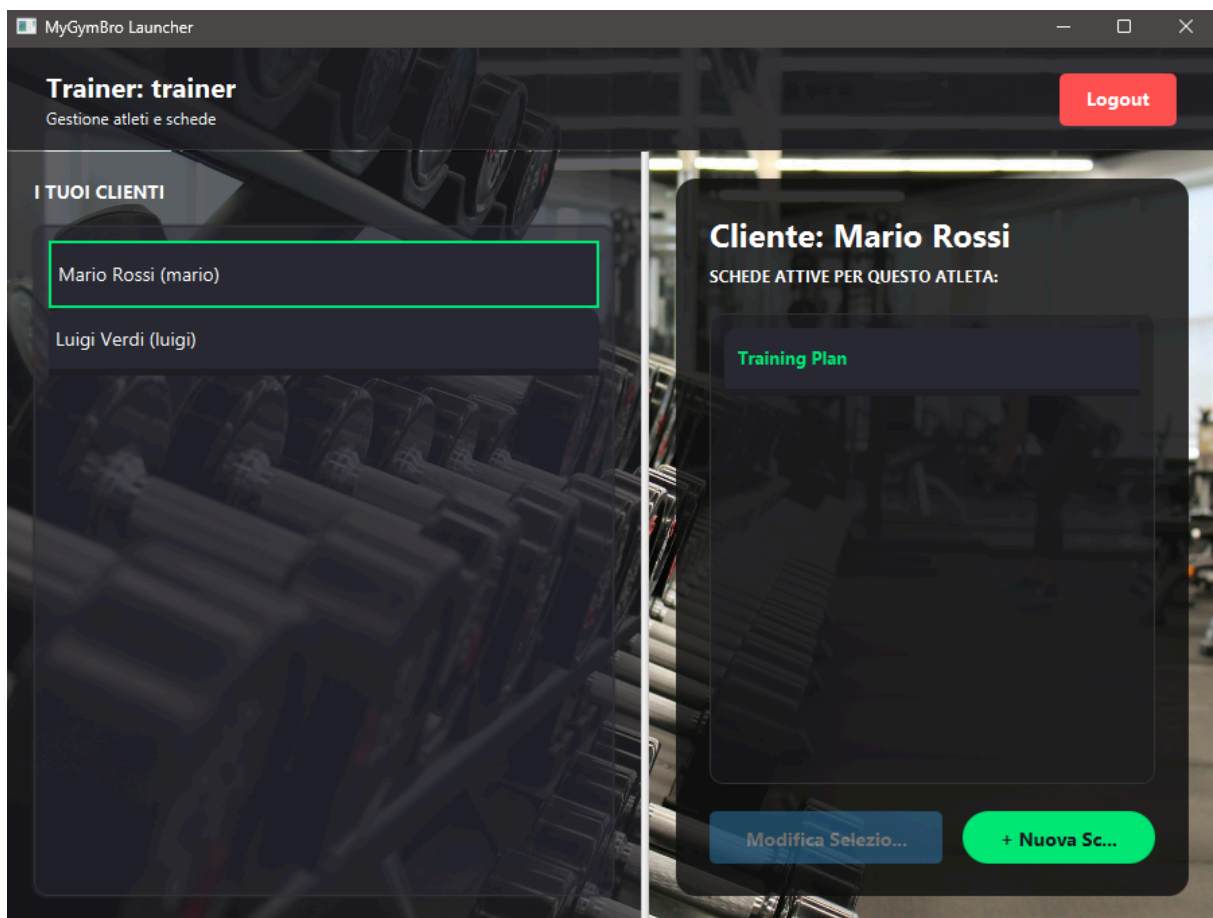
AGGIUNGI

ESERCIZIO	MUSCOLO	SETS	REPS	REST (s)	ELIMINA
assisted chest dip (kneeling)	CHEST	3	10	90	X

ANNULLA

SALVA E ASSEGNA SCHEDA

Personal Trainer's Home:

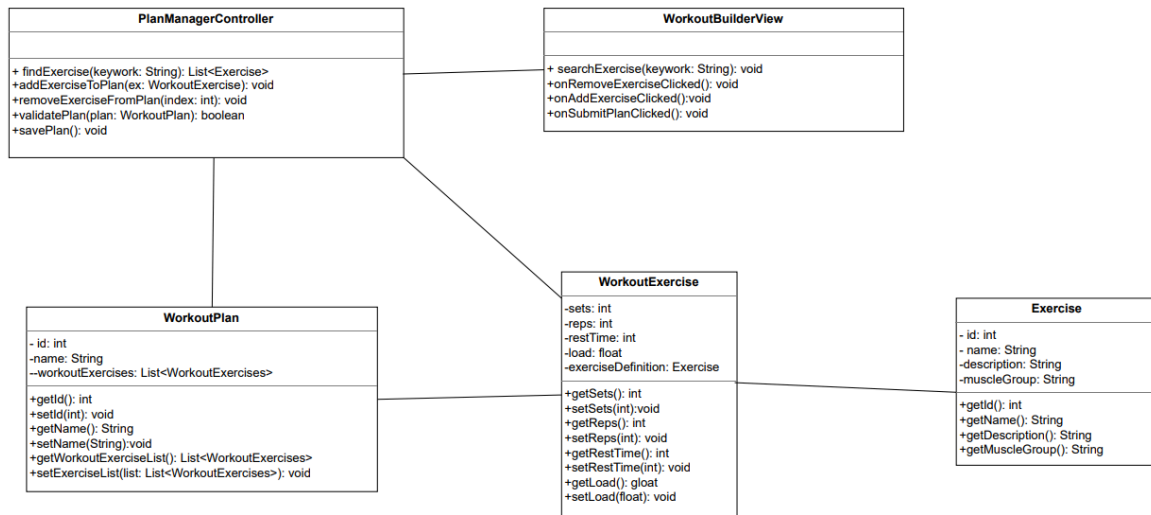


3 Design

3.1 Class Diagram

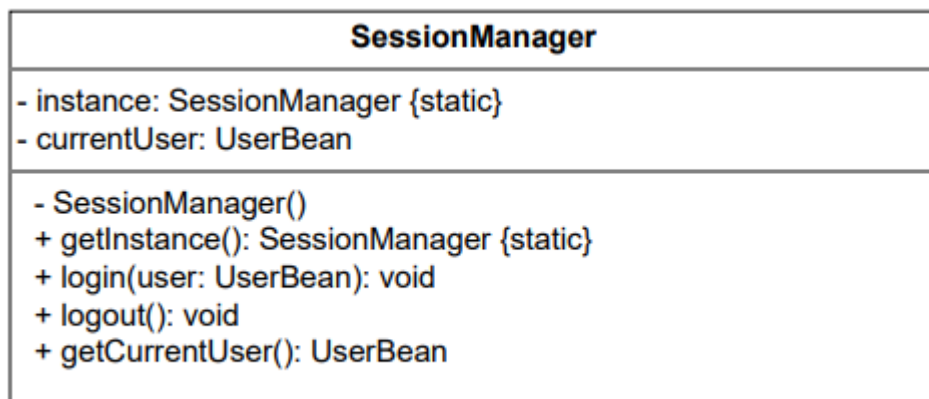
3.1.1 VOPC

Create WorkoutPlan:

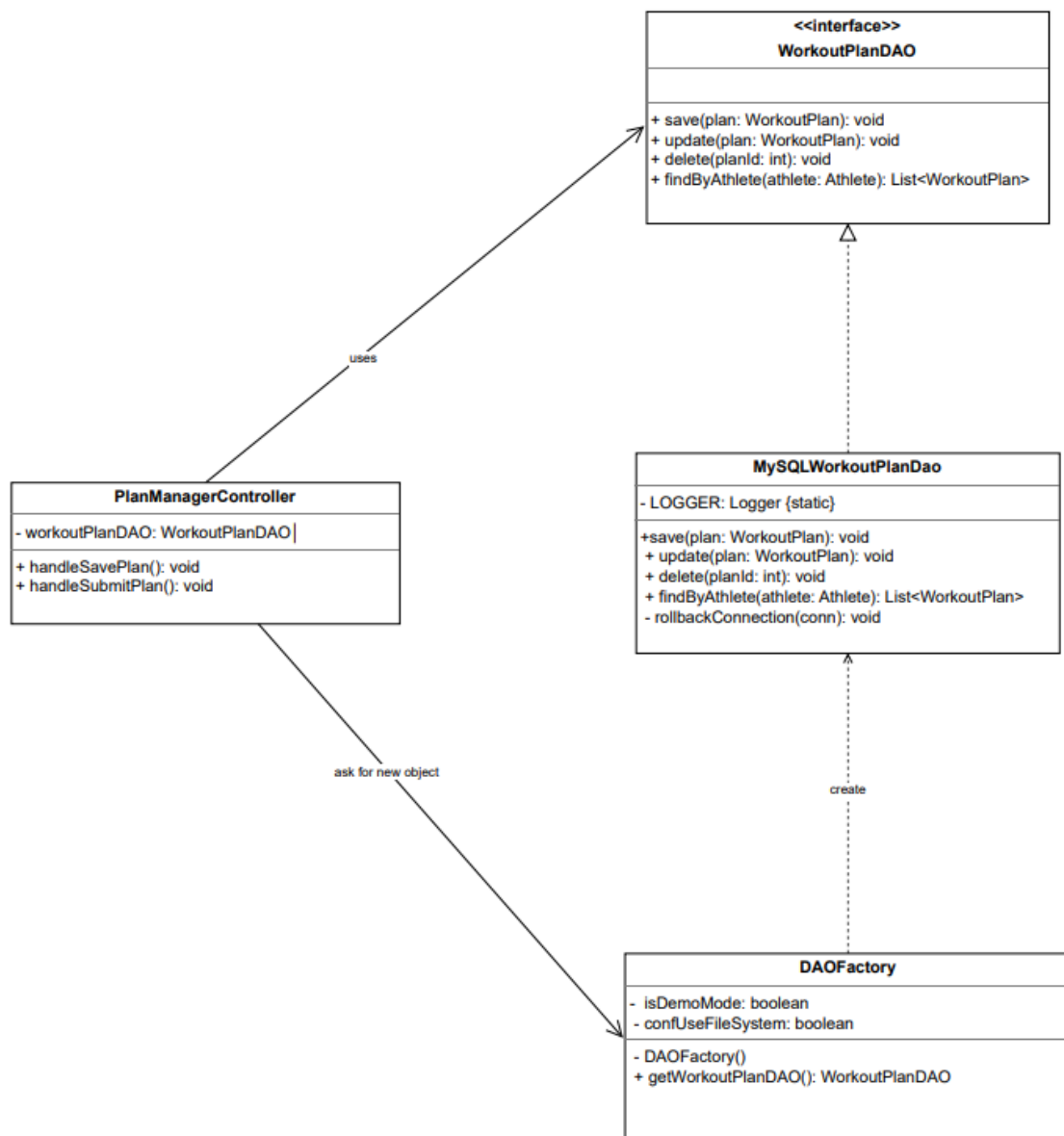


3.1.2 Design-Level Diagram

Singleton:



Factory Method:



3.2 Design patterns

Singleton:

In MyGymBro, we've adopted the Singleton Pattern for managing user sessions via the SessionManager class, maintaining a shared, centralized state throughout the app's execution. This allows us to consistently access logged-in user information from any part of the system, avoiding duplication or inconsistencies. Instance uniqueness, guaranteed by `getInstance()`, allows all controllers to reference the same session without explicit object transfers, making the code cleaner and more modular.

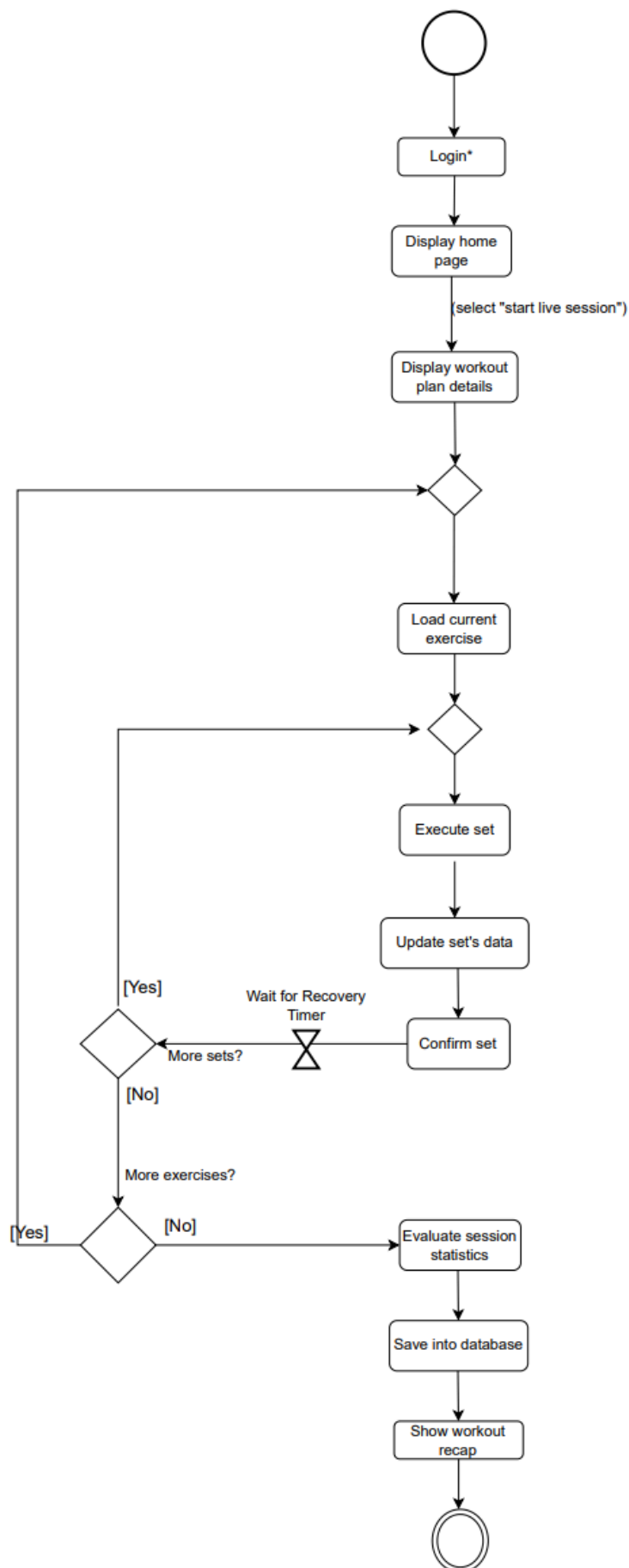
Factory Method Pattern:

The Factory Method pattern is implemented to manage the instantiation of the Data Access Objects (DAOs), ensuring a clean separation between the business logic and the persistence layer. In this architecture, the **DAOFactory** acts as the centralized creator, responsible for

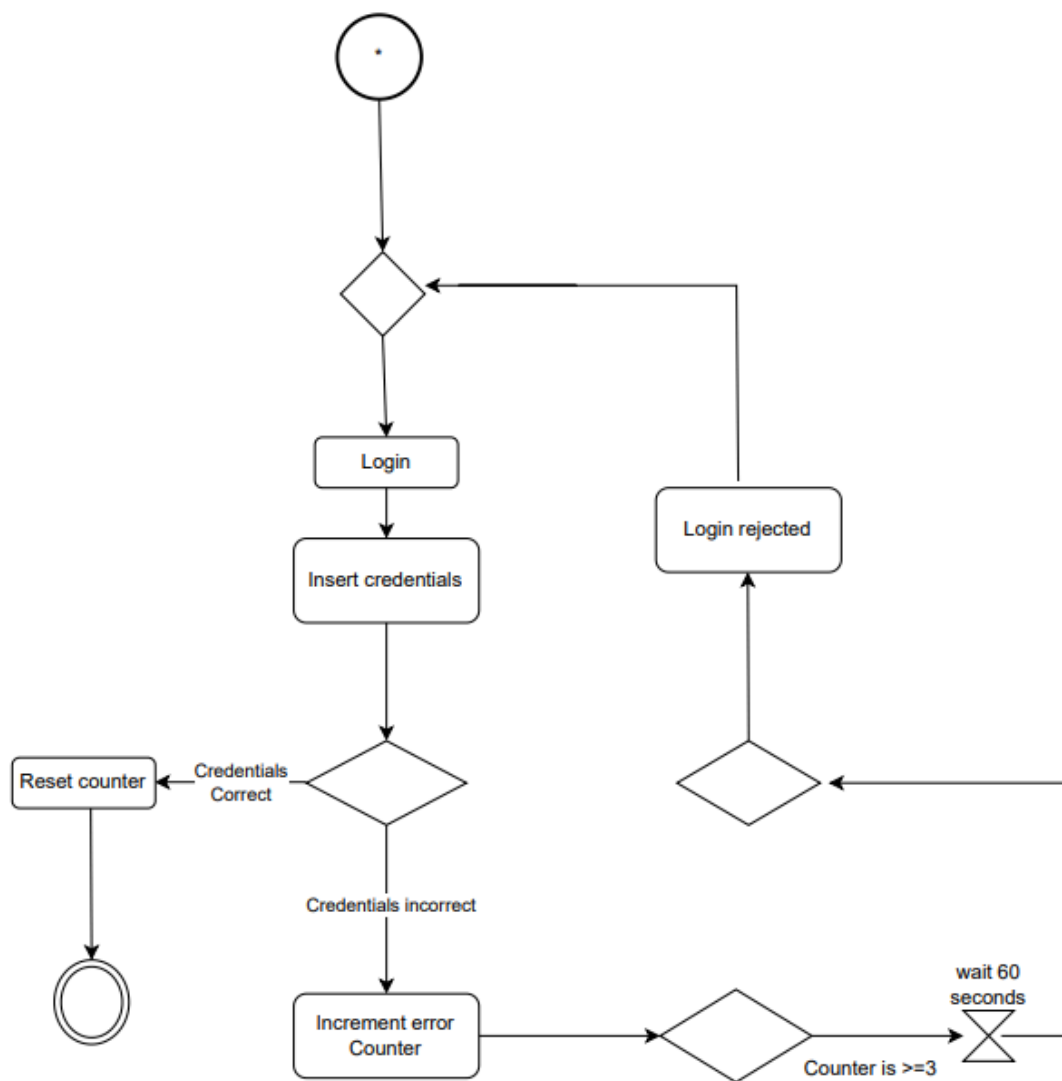
generating concrete instances of the DAO interfaces based on the required persistence strategy

3.3 Activity Diagram

UC: Start live session

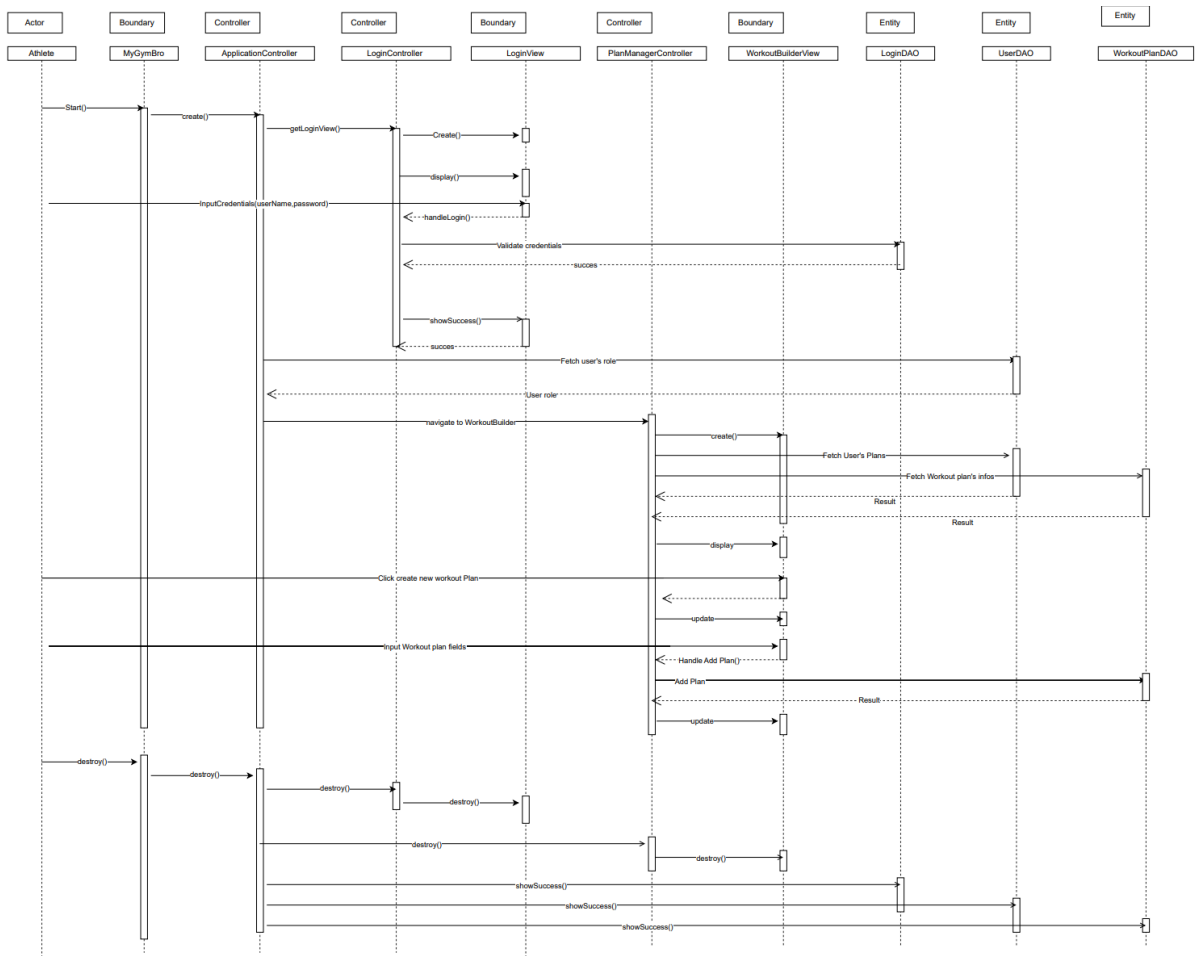


UC: Login



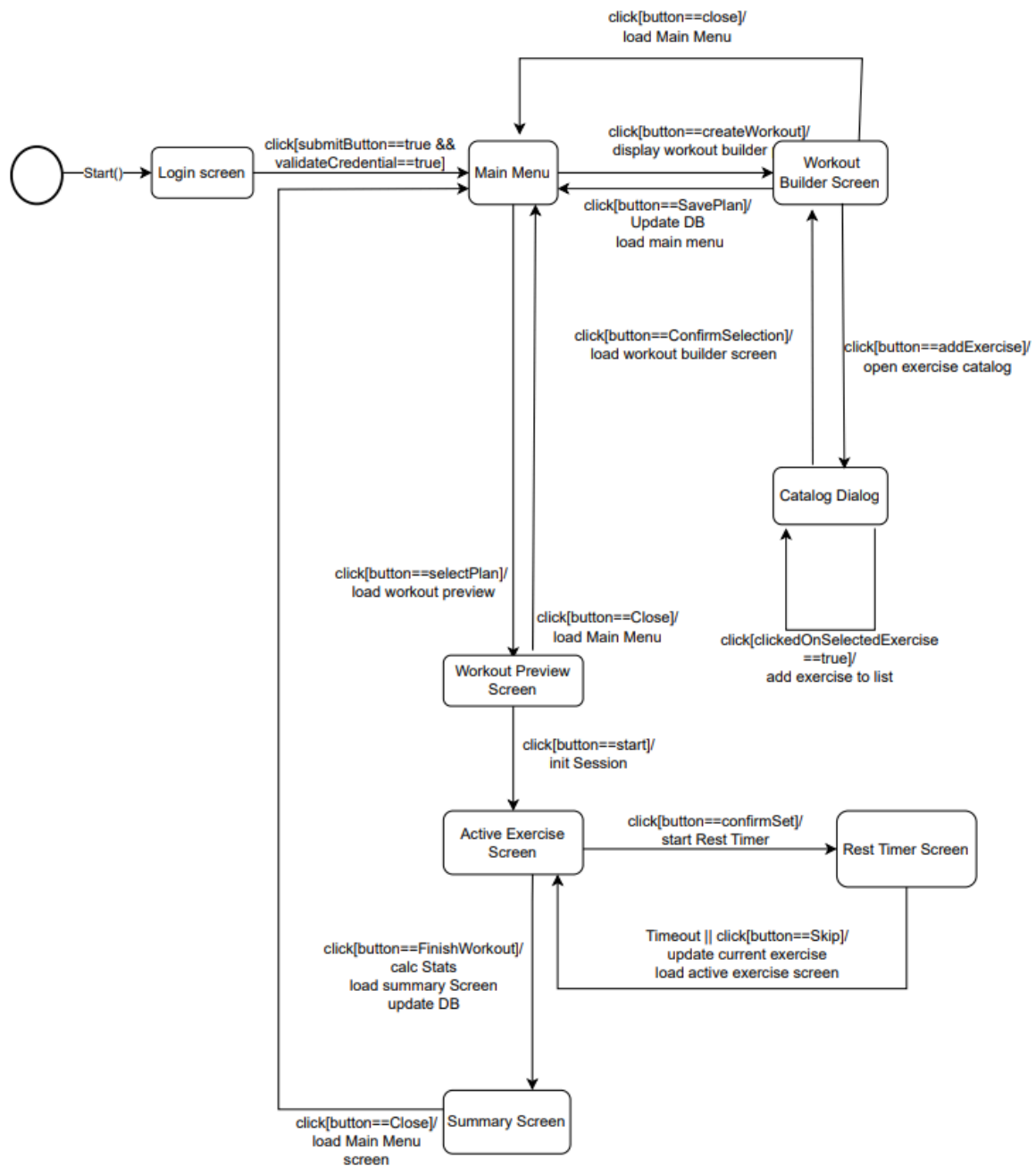
3.4 Sequence Diagram

UC: Create Workout Plan



3.5 State Diagram

The following diagram shows the main functionalities of the system in case of an Athlete User



4 TESTING

WorkoutPlanTest

This test class verifies the core functionality of the workout creation module by simulating the assembly of a new training plan. Using the AAA (Arrange-Act-Assert) pattern, the test ensures that the WorkoutPlanBean correctly aggregates WorkoutExerciseBean instances and

maintains data integrity for the plan's name and exercise count. By validating the association between plans and exercises, the test guarantees that the system's primary business logic for trainers and athletes is reliable and free from regression.

ExerciseValidationTest

The ExerciseValidationTest class focuses on data integrity by validating the input parameters for specific exercises, such as sets, repetitions, and rest intervals. It covers multiple scenarios, including standard valid data, minimum acceptable thresholds, and realistic training ranges. These tests ensure that the system prevents the entry of illogical or negative values, maintaining high quality in the training plans generated by users and aligning with the functional requirements for error-free data entry.

SessionTest

This test class evaluates the security and persistence of the user session managed by the SessionManager singleton. The test simulates a complete lifecycle—from user authentication and session assignment to a total logout—ensuring that the UserBean is correctly stored and subsequently cleared from memory. Validating the session transition is crucial for preventing unauthorized access and ensuring that the system's state remains consistent across different application views, as specified in the login use case.

5 GitHub Repository

<https://github.com/dadoexe/MyGymBro-final>

6 SonarCloud

https://sonarcloud.io/summary/new_code?id=dadoexe_MyGymBro-final

7 Presentation Video

<https://youtu.be/5TNnuL7RuWA>